

Hash-Linear Probin ▾

HASH-LINEAR PROBING

Write a program to create a hash table of size 10, insert the elements 25, 35, 15, 45, 95. If any collisions occurred, then resolve collisions using linear probing. Finally display the hash table after each insertion.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int hash[10];
6     int a[5] = {25, 35, 15, 45, 95};
7     int i, j, index;
8
9     for (i = 0; i < 10; i++)
10         hash[i] = -1;
11
12     for (i = 0; i < 5; i++)
13     {
14         index = a[i] % 10;
15
16         while (hash[index] != -1)
17         {
18             index = index + 1;
19
20             if (index == 10)
21                 index = 0;
```

OUTPUT

Absolute system paths (e.g. C:\) are not allowed.

Hashing-Delete



HASHING-DELETE

Write a program to perform the following operations using hashing: Insert an element, search an element and delete an element

```
1 #include <stdio.h>
2
3 #define SIZE 10
4
5 int hashTable[SIZE];
6
7 void insert(int key) {
8     int index = key % SIZE;
9
10    while (hashTable[index] != -1)
11        index = (index + 1) % SIZE;
12
13    hashTable[index] = key;
14 }
15
16 int search(int key) {
17     int index = key % SIZE;
18     int start = index;
19
20    while (hashTable[index] != -1) {
21        if (hashTable[index] == key)
```

OUTPUT

Hash Table:

```
0 : -1
1 : -1
2 : -1
3 : -1
4 : -1
5 : 25
6 : 35
7 : 15
8 : 45
```

Hash-Store & Delete ▾

HASH-STORE & DELETE

Develop a program using hashing to store employee details, search employee details using employee ID and delete employee records

```
1 #include <stdio.h>
2
3 #define SIZE 10
4
5 struct Employee {
6     int id;
7     char name[30];
8     float salary;
9 };
10
11 struct Employee hashTable[SIZE];
12
13 void insert(int id, char name[], float salary) {
14     int index = id % SIZE;
15
16     while (hashTable[index].id != -1)
17         index = (index + 1) % SIZE;
18
19     hashTable[index].id = id;
20     sprintf(hashTable[index].name, "%s", name);
21     hashTable[index].salary = salary;
```

OUTPUT

Employee Records:

101 Arun 25000.00
102 Bala 30000.00
111 Kumar 28000.00

Employee found: 102 Bala 30000.00

Employee deleted successfully

Employee Records:

101 Arun 25000.00

Quadratic Probing ▾

QUADRATIC PROBING

Write a program to insert the keys 12, 22, 32, 42, 52 into a hash table using quadratic probing and display the final hash table.

```
1 #include <stdio.h>
2
3 #define SIZE 10
4
5 int main() {
6     int hash[SIZE];
7     int keys[] = {12, 22, 32, 42, 52};
8     int n = 5;
9     int i, j, index;
10
11     for (i = 0; i < SIZE; i++)
12         hash[i] = -1;
13
14     for (i = 0; i < n; i++) {
15         index = keys[i] % SIZE;
16         j = 0;
17
18         while (hash[index] != -1) {
19             j++;
20             index = (keys[i] % SIZE + j * j) % SIZE;
21         }
```

OUTPUT

```
Index 0 : -1
Index 1 : 42
Index 2 : 12
Index 3 : 22
Index 4 : -1
Index 5 : -1
Index 6 : 32
Index 7 : -1
Index 8 : 52
Index 9 : -1
```

Hash-Separate Che ▾

HASH-SEPARATE CHAINING

Write a program for constructing a hash table of size 5. Insert the values 10, 15, 20, 25, 30, 35. Handle collisions using separate chaining. Display the final hash table.

```
1 #include <stdio.h>
2
3 #define SIZE 5
4
5 int main() {
6     int hash[SIZE][10];
7     int count[SIZE] = {0};
8     int values[] = {10, 15, 20, 25, 30, 35};
9     int n = 6;
10    int i, index;
11
12    /* Initialize hash table */
13    for (i = 0; i < SIZE; i++)
14        count[i] = 0;
15
16    /* Insert using separate chaining */
17    for (i = 0; i < n; i++) {
18        index = values[i] % SIZE;
19        hash[index][count[index]] = values[i];
20        count[index]++;
21    }
```


OUTPUT

```
Final Hash Table:  
Index 0: 10 -> 15 -> 20 -> 25 -> 30 -> 35  
Index 1: NULL  
Index 2: NULL  
Index 3: NULL  
Index 4: NULL
```